Fundamentos de Java, Arquitectura y Tipos de Datos Primitivos

Guía de Trabajo Autónomo:

1. Información General y Metodología de Trabajo

Bienvenido a esta Guía de Trabajo Autónomo. Este espacio de aprendizaje ha sido diseñado bajo una metodología de trabajo independiente con una duración estimada de **3 horas**. El propósito principal de esta actividad no es escribir programas ejecutables de inmediato, sino construir una base conceptual sólida, rigurosa e ingenieril sobre el ecosistema Java.

A lo largo de este taller, investigarás, analizarás y estructurarás el conocimiento necesario para elaborar una **Ficha Técnica de Java ("Java Datasheet")**. Este documento teórico consolidará la historia, arquitectura, tipos de datos primitivos, gestión de memoria y convenciones de sintaxis del lenguaje.

OBJETIVO CLAVE DE APRENDIZAJE Comprender la arquitectura de ejecución de Java, la filosofía de portabilidad y la gestión teórica de memoria mediante el análisis de tipos de datos primitivos y convenciones sintácticas, consolidando una Ficha Técnica ("Java Datasheet") sin requerir la escritura de código ejecutable.

Requisitos, Materiales y Marco de Referencia

Para completar con éxito esta guía autónoma, asegúrate de contar con los siguientes elementos:

- **Línea de comandos:** Terminal (macOS/Linux) o Símbolo del sistema / PowerShell (Windows).
- **Kit de Desarrollo:** JDK (Java Development Kit) versión 17 o 21 LTS instalado.
- **Entorno de Desarrollo Integrado:** IntelliJ IDEA Community Edition instalado.
- **Herramienta Gráfica:** Software para creación de mapas conceptuales (ej. Draw.io, Miro, Lucidchart) o papel y lápiz.
- **Marco Especificación Técnica:** Las investigaciones e indagaciones teóricas requeridas en esta guía toman como fundamento la especificación oficial del lenguaje (*Java Language Specification* - JLS para Java 17 y 21 LTS).

Entrega

El producto final individual de esta guía de trabajo es la Ficha Técnica ("Java Datasheet"), la cual deberá ser ensamblada siguiendo las pautas de la Parte 3 y entregada exclusivamente en **formato PDF**.

Con los objetivos y materiales claros, es momento de iniciar la primera hora de trabajo adentrándonos en las raíces históricas, los pilares de diseño y la arquitectura interna que impulsa a Java.

2. Historia, Filosofía, Arquitectura y Verificación del Entorno

2.1. Investigación y Estructura para el Mapa Conceptual

Durante esta primera fase, realizarás una investigación teórica sobre los orígenes y la arquitectura de Java. Con la información recolectada, diseñarás un mapa conceptual (digital o analógico) que sintetice de manera clara los tres ejes temáticos fundamentales descritos a continuación:

1. **Orígenes e Historia:**
   - Creación por **James Gosling** y el equipo de **Sun Microsystems**.
   - Proceso de adquisición y evolución bajo **Oracle Corporation**.
2. **Filosofía de Diseño:**
   - El principio **WORA** (*Write Once, Run Anywhere* / "Escríbelo una vez, ejecútalo en cualquier lugar").
   - Significado práctico de la portabilidad frente a lenguajes compilados directamente a código máquina nativo.
3. **Arquitectura Interna y Ciclo de Ejecución:**
   - **JDK (Java Development Kit):** Conjunto completo de herramientas de desarrollo (compilador, utilidades, entorno).
   - **JRE (Java Runtime Environment):** Entorno necesario para la ejecución que contiene las librerías estándar.
   - **JVM (Java Virtual Machine):** Máquina virtual encargada de interpretar y compilar en tiempo de ejecución (*JIT Compilation*) el código intermedio.
   - **Bytecode (Archivos `**.class**`):** Formato intermedio e independiente de la plataforma generado por el compilador (`javac`).

Plantilla Conceptual Sugerida para el Mapa

Utiliza la siguiente lista jerárquica como modelo estructural para organizar los nodos y relaciones en tu mapa conceptual:

- **Ecosistema Java**
  - **1. Historia y Evolución**
    - Creador: James Gosling (Sun Microsystems)
    - Evolución actual: Oracle Corporation
  - **2. Filosofía de Diseño**
    - WORA (*Write Once, Run Anywhere*)
      - Independencia del sistema operativo
      - Compilación a formato neutro (Bytecode)
  - **3. Arquitectura del Entorno**
    - **JDK** (Contiene herramientas de desarrollo + JRE)
      - Compilador (`javac`)
    - **JRE** (Contiene librerías de tiempo de ejecución + JVM)
      - Librerías de clases estándar
    - **JVM** (Motor de ejecución)
      - Carga de archivos `.class` (Bytecode)
      - Traducción a código máquina nativo

2.2. Guía Paso a Paso para la Verificación del Entorno (Sin Código)

Antes de avanzar a la investigación de datos en memoria, debes confirmar que las herramientas de bajo nivel del sistema están correctamente instaladas y configuradas. Realiza las siguientes comprobaciones directamente en la terminal de tu equipo:

1. **Comprobación del Compilador (`**javac**`):** Abre la terminal y ejecuta el comando:

```
javac -version
```

Resultado esperado: La terminal debe retornar la versión del compilador de Java (ej. `javac 17.x.x` o `javac 21.x.x`). Registra este resultado.

1. **Comprobación del Entorno de Ejecución (`**java**`):** En la misma terminal, ejecuta el comando:

```
java -version
```

Resultado esperado: Confirmación de la versión de la máquina virtual (JVM), verificando que corresponda a una versión LTS (17 o 21). Registra este resultado.

1. **Confirmación de Entorno IDE:** Abre la aplicación **IntelliJ IDEA Community Edition** y verifica visualmente que la pantalla de bienvenida detecte la instalación del JDK del sistema.

ADVERTENCIA: En esta fase del proceso pedagógico **NO** debes crear proyectos de código, abrir clases `.java` ni escribir programas ejecutables (como el tradicional *Hello World*). La verificación del entorno es un paso puramente operativo de configuración del sistema.

2.3. Transición Pedagógica

Comprender la arquitectura interna de la JVM y la función del Bytecode nos permite visualizar cómo Java gestiona los recursos en ejecución; este entendimiento es indispensable para analizar, en la siguiente hora, cómo los tipos de datos primitivos optimizan el uso de la memoria.

3. Investigación de Tipos de Datos Primitivos y Buenas Prácticas

3.1. Matriz de Investigación Teórica de Tipos Primitivos

Java dispone de 8 tipos de datos primitivos integrados a nivel de lenguaje. Durante esta hora de trabajo autónomo, investigará la especificación técnica de cada uno de ellos en la documentación de la JLS y completará la siguiente matriz dentro de la Sección B de su **Java Datasheet**:

Tipo Primitivo	Tamaño en Memoria (Bits/Bytes)	Rango de Valores Permitido	Valor por Defecto	Sintaxis Teórica de Declaración
`byte`	*[Completar]*	*[Completar]*	*[Completar]*	`byte nombreVariable;`
`short`	*[Completar]*	*[Completar]*	*[Completar]*	`short nombreVariable;`
`int`	*[Completar]*	*[Completar]*	*[Completar]*	`int nombreVariable;`
`long`	*[Completar]*	*[Completar]*	*[Completar]*	`long nombreVariable;`
`float`	*[Completar]*	*[Completar]*	*[Completar]*	`float nombreVariable;`
`double`	*[Completar]*	*[Completar]*	*[Completar]*	`double nombreVariable;`
`char`	*[Completar]*	*[Completar]*	*[Completar]*	`char nombreVariable;`

`boolean`	*[Completar]*	*[Completar]*	*[Completar]*	`boolean nombreVariable;`

3.2. Criterios de Selección y Buenas Prácticas Reales

Para completar la Sección B de tu **Java Datasheet**, analiza, investiga y responde con tus propias palabras los siguientes tres problemas y preguntas orientadoras de diseño técnico:

1. **Investigación de Precisión Financiera frente a Tipos Flotantes:**
   - *Pregunta de investigación:* ¿Por qué la especificación del estándar de coma flotante IEEE 754 (utilizado por `float` y `double`) provoca imprecisiones de redondeo en operaciones aritméticas binarias, haciendo que estos tipos sean totalmente inadecuados para el cálculo monetario o financiero?
   - *Desafío conceptual:* Investiga y fundamenta cuál es la alternativa teórica orientada a objetos (clase estándar de Java) recomendada profesionalmente para manejar valores monetarios y operaciones de precisión decimal exacta.
2. **Criterio de Selección de Enteros (`**int**` vs `**long**`):**
   - *Pregunta de investigación:* ¿Cuál es el límite en bits y rango numérico del tipo `int` (aproximadamente 2.14 \times 10^9) y bajo qué escenarios numéricos específicos de la industria (ej. IDs numéricos de bases de datos masivas, marcas de tiempo en milisegundos *Unix Timestamp*) se justifica el salto al tipo `long` de 64 bits?
3. **Análisis del Tipo Booleano en Arquitectura de Software:**
   - *Pregunta de investigación:* ¿Cuáles son los casos de uso idóneos del tipo `boolean` dentro del flujo de control de un programa y cuál es su comportamiento semántico al trabajar como bandera (*flag*) de estado o evaluador de expresiones lógicas?

3.3. Transición Pedagógica

La correcta elección teórica de los tipos de datos en memoria se complementa de forma directa con las normas formales de sintaxis, la definición de inmutabilidad y las convenciones estandarizadas de nombres que estructurarán la fase final de nuestra investigación.

4. Consolidación del 'Java Datasheet' Teórico y Convenciones

4.1. Reglas Sintácticas, Constantes y Convenciones

Sintetiza y registra en la Sección C de tu **Java Datasheet** los estándares sintácticos fundamentales del lenguaje antes de realizar el ensamblado final:

- **Inmutabilidad y Definición de Constantes:** Investiga el rol de la palabra reservada `final` en Java aplicada a variables de tipos primitivos y explica conceptualmente por qué la inmutabilidad favorece la seguridad y consistencia en el diseño de software.
- **Estilos de Nomenclatura Estandarizados:** Define y ejemplifica los dos estándares de nombres dominantes en Java:
  - `camelCase` (Lower Camel Case): Reglas de uso para **variables** y **métodos**.
  - `UPPER_SNAKE_CASE`: Reglas de uso para **constantes** asociadas a la palabra reservada `final`.

Comparación Analítica de Formatos de Salida Teóricos

Analiza los mecanismos conceptuales que ofrece Java para enviar información a la salida estándar y sintetiza sus diferencias completando la estructura de la siguiente tabla comparativa en tu Ficha Técnica:

Característica	`System.out.println()`	`System.out.printf()`
Propósito Principal	Imprimir una línea de texto simple o concatenada, agregando un salto de línea automático al final.	Imprimir texto utilizando cadenas de formato con especificadores para estructurar la salida.
Flexibilidad de Formato	Básica. Requiere concatenación manual mediante el operador `+`.	Alta. Permite definir precisión decimal, alineación, anchos de campo y formatos especiales.
Casos de Uso Indicados	Mensajes de estado sencillos, depuración rápida o impresiones lineales sin requerimiento estético.	Reportes tabulares, formateo de valores monetarios o alineación precisa de cadenas y números.

4.2. Estructura e Instrucciones de Ensamblado del 'Java Datasheet'

Consolida todo el trabajo investigativo realizado en un único documento titulado **"Java Datasheet"**. Estructura tu documento respetando rigurosamente el siguiente esquema y plantilla visual:

```
======================================================================
                  JAVA DATASHEET – FICHA TÉCNICA
======================================================================

SECCIÓN A: HISTORIA, FILOSOFÍA Y ARQUITECTURA
----------------------------------------------------------------------
1. Diagrama / Mapa Conceptual de Arquitectura:
   [Inserción de la imagen del mapa conceptual o plantilla jerárquica
   que incluya: Gosling/Sun/Oracle, WORA, JDK, JRE, JVM y Bytecode]

2. Síntesis Teórica del Motor de Ejecución:
   [Explicación con tus palabras sobre cómo se relaciona el compilador
   javac con el Bytecode (.class) y la máquina virtual JVM]

SECCIÓN B: TIPOS DE DATOS PRIMITIVOS Y GESTIÓN DE MEMORIA
----------------------------------------------------------------------
1. Matriz Teórica Completada:
   [Inserción de la Tabla de los 8 Tipos Primitivos con sus datos de
   bits, rangos, valores por defecto y sintaxis de declaración]

2. Justificaciones de Ingeniería de Selección de Tipos:
   a) Análisis de Precisión Financiera e IEEE 754 (y alternativa BigDecimal)
   b) Criterios de Selección: int vs long
   c) Aplicación del tipo boolean en control de flujo
```

```
SECCIÓN C: SINTAXIS, CONVENCIONES Y FORMATO DE SALIDA
------------------------------------------------------------------------
1. Ficha de Reglas Sintácticas:
   - Uso de 'final' e Inmutabilidad
   - Convenciones de Nomenclatura: camelCase vs UPPER_SNAKE_CASE (con ejemplos)

2. Tabla Comparativa de Salida Estándar:
   [Tabla comparativa entre System.out.println() y System.out.printf()]

SECCIÓN D: VERIFICACIÓN TÉCNICA DEL ENTORNO
------------------------------------------------------------------------
1. Registro de Comandos en Terminal:
   - Salida del comando: javac -version
   - Salida del comando: java -version
   - Confirmación visual de IDE IntelliJ IDEA Community Edition
========================================================================
```

4.3. Lista de Chequeo y Autoevaluación

Antes de exportar tu documento final a formato **PDF**, verifica de manera autónoma el cumplimiento de cada uno de los siguientes criterios marcando las casillas correspondientes:

- [ ] **Contenido 100% Teórico:** Confirmar que el documento final **no** contiene código ejecutable escrito ni proyectos compilados.
- [ ] **Mapa Conceptual Integrado:** Validar que el mapa conceptual incluye a James Gosling, Sun/Oracle, la filosofía WORA y la relación explícita JDK / JRE / JVM / Bytecode.
- [ ] **Matriz de Tipos Primitivos Completa:** Asegurar que los 8 tipos primitivos (`byte` , `short` , `int` , `long` , `float` , `double` , `char` , `boolean`) cuentan con sus bits, rangos, valores por defecto y sintaxis completados.
- [ ] **Preguntas de Investigación Respondidas:** Verificar que se redactaron las respuestas a los tres problemas analíticos (IEEE 754/financiero, `int` vs `long` , y uso de `boolean`).
- [ ] **Ficha de Sintaxis y Convenciones:** Confirmar la explicación de la inmutabilidad con `final` , los ejemplos de `camelCase` / `UPPER_SNAKE_CASE` y la tabla de `println` vs `printf` .
- [ ] **Verificación del Entorno:** Incluir la confirmación textual de las salidas de `javac -version` y `java -version` obtenidas en la terminal.
- [ ] **Formato de Entrega:** Asegurar que la estructura concuerda con la plantilla del *Java Datasheet* y que está lista para ser exportada a **PDF**.